Assignment - 06

Aim:
Write a program to implement polynomial operation using a Circular linked list. The program should support the following operations:
1) Create a polynomial by inserting terms (coefficient and exponent),
2) Display the polynomial in a readable mathematical form.
3) Add two polynomials and display the resultant polynomial.

Objectives:
1) To implement a polynomial represent using a Circular Linked List by dynamically creating and storing polynomial terms (coefficient and exponent),
2) To perform fundamental polynomial operations such as creating, displaying and adding two polynomials and display the resultant polynomial,

Objectives:
1) To implement a polynomial representation using a Circular Linked List by dynamically creating, display, and adding two polynomials using Circular Linked List techniques,
2) To understand the advantage of Circular Linked lists in managing dynamic data structures and efficiently handling polynomial arithmetic by combining like terms,

Theory

Implemented as:
- Platform
  - 64 bit Open source Linux or its derivatives.
  - Open Source C Programming tool like gcc / Eclipse Editor

① Pseudo Code :-

START

1) CREATE_NODE:
Allocate a node and store coefficient and exponent.

2) INSERT_TERM:
If list is empty, create a circular node.
If exponent already exists, add coefficients.
Otherwise, insert the term in descending exponent order.
Delete the term if its coefficient becomes zero.

3) CREATE POLYNOMIAL:-
Input number of terms.
Read coefficient and exponent for each term.
Insert each term into the circular linked list.

4) Display_Polynomial:
If list is empty, display 0.

5) Add_Polynomials:
Create an empty result list.
Transverse the first polynomial & insert its terms.
Transverse the second polynomial and insert its terms.
Combine terms having equal exponents.
Return the result list.

6) FREE_POLYNOMIAL
Transverse the circular list and free every node set the head pointer to NULL.

7) MAIN:
Create two polynomials

Display both polynomials.
Add them and display the result.
Free memory allocated to all three lists.

STOP

- Time Complexity :-

Operation

1) Create a node — => O(1)
2) Insert a term => O(n)
3) Create a polynomial with n input terms. => $O(n^2)$
4) Display polynomial => O(n)
5) Add two polynomials => $O((n+m)^2)$
6) Free a polynomial => O(n)

⊙ Conclusion :-
The program successfully implements polynomial creation display and addition using a circular linked list, demonstrating efficient dynamic memory mangent and polynomial manipulation

- FAQs

1) Why is a Circular Linked List used to represent polynomials instead of a Linear Linked List?
Ans A Circular Linked List allows continous transversal because the best node points to the first node

2) What information is stored in each node of the circular Linked List?

Ans Each node stores the coefficient, exponent and a pointer to the next node.

3) What happens if one of the polynomial's is empty during the addition operation?

Ans) If one polynomial is empty, the result is the other polynomial. If both are empty, the result is an empty polynomial, displayed as 0.